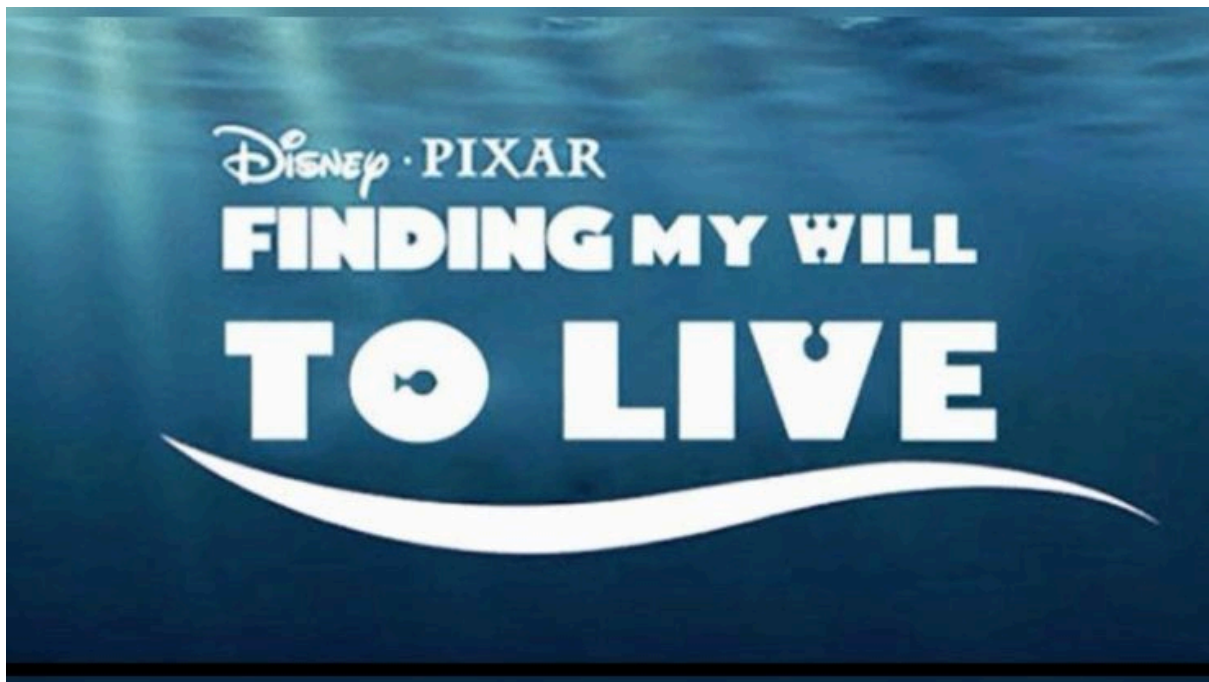


Course: Individual Software Development Process' 2026

Software Proposal

By Unpaid Internship

Chosen Irl Challenge: Project C: Department Information &
Communication Hub



Chutikarn Stenger	6810545573
Dhechid Phumpaka	6810545603
Varissara Arayavilaipong	6810545891
Wiran Watanathira-anan	6810545913

Table of Contents

1. Target Users.....	1
2. The System Goal.....	1
3. Goal Refinement via KAOS:.....	1
4. Use Case Diagram.....	2
5. Use Stories.....	3
6. User Requirement Specification.....	6
7. Activity Diagram.....	7
8. System Requirement Specification.....	11
9. Software Architecture.....	12
Rationale.....	12
10. Data Storage Technology.....	10
Rationale.....	10
11. Development Environment.....	11
Rationale.....	11
12. Sequence Diagrams for All SRS.....	13
13. Traceability Matrix.....	17

1. Target Users

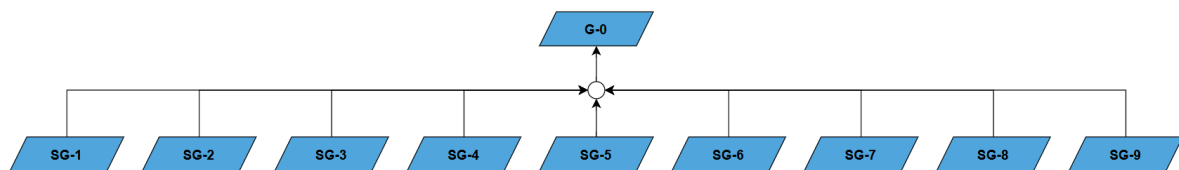
- **Student:** Must log in via Google OAuth2 (restricted to @ku.th email addresses) before accessing the system. Can view announcements, FAQs, schedules, contacts, resource links, search by keyword, and filter by tag.
- **TA/Lecturers:** Must log in via Google OAuth2. Create, edit, publish, and archive content and assign target tags before publishing announcement.
- **Department system administrator:** Must log in via Google OAuth2. Manage overall system structure, user access, and maintain the tag list used for content targeting.

2. The System Goal

G-0: Maintain a single, accessible hub which is accessible only to @ku.th account holders for every announcement and information to reduce confusion for students and repeated workload for TAs and Lecturers.

At this moment, students rely on a mix of webpages, scattered LMS announcements, chat groups, emails, and printed notices, causing confusion about which information is current and valid. Moreover, this problem also creates repeated workload for TAs and lecturers who have to answer the same questions from students. This iteration adds two dimensions beyond the original five content types: **(1) restricting system access to department members** via Google OAuth2, and **(2) keyword search** across all content, complementing the tag-based filtering already designed. Staff/TA retain full publish/archive authority without a review step.

3. Goal Refinement via KAOS:



- **Sub-Goal 1 (SG-1)**
Description: Achieve: Provide students with current department information.
Rationale: Students need accurate information before asking TAs or lecturers.
- **Sub-Goal 2 (SG-2)**
Description: Maintain: complete content details per type
Rationale: Each content type has its own required fields.
- **Sub-Goal 3 (SG-3)**
Description: Achieve: allow only authorised users to publish updates.
Rationale: Uncontrolled edits could introduce incorrect or conflicting information.
- **Sub-Goal 4 (SG-4)**
Description: Avoid: outdated content being shown to students.

Rationale: Stale or incomplete information recreates the confusion the hub is meant to solve

- **Sub-Goal 5 (SG-5)**

Description: Maintain: manageable content archive.

Rationale: Archived items should no longer appear publicly but may be retained for staff and reference.

- **Sub-Goal 6 (SG-6)**

Description: Achieve: let student filter content by tag

Rationale: Reduced clutter when the hub is used departmentwide across multiple audiences.

- **Sub-Goal 7 (SG-7)**

Description: Maintain: an administrator controlled tag list.

Rationale: Supports new courses overtime without requiring code changes.

- **Sub-Goal 8 (SG-8)**

Description: Achieve: restrict system access to @ku.th account holders.

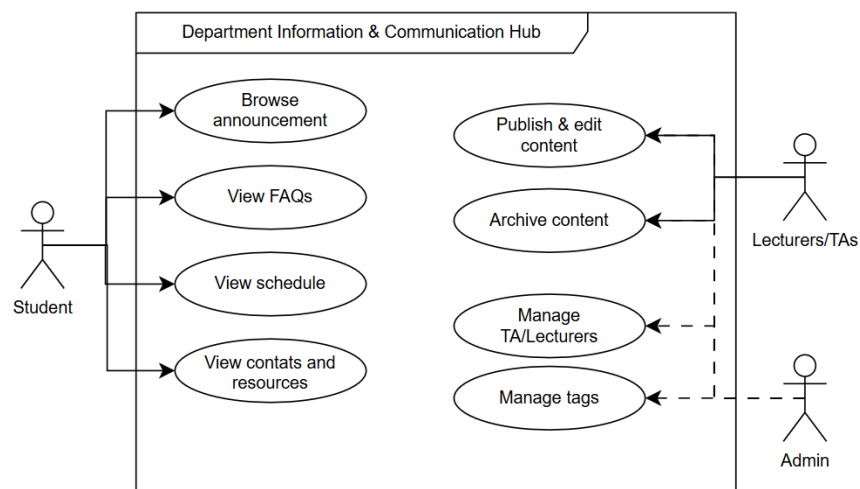
Rationale: Prevents outside parties from accessing department information.

- **Sub-Goal 9 (SG-9)**

Description: Achieve: let users search content by keyword across all types.

Rationale: Complements tag filtering; confirmed as a requirement by the stakeholder.

4. Use Case Diagram



5. User Stories

User Story ID	Description	Acceptance Criteria
US-1	As a student, I want to view current announcements so I do not miss any updates.	The announcement list displays title, published date, and author for each item, sorted by most recent first. Only items with status == published are shown. Items with status == archived or draft do not appear in this list under any condition.
US-2	As a student, I want to search FAQs so I can find answers without asking a TA.	A search field and category filter are available. Typing a keyword returns information that contains a match, case-insensitive. Selecting a category shows only entries tagged with that category. Each result displays the full question and answer.
US-3	As a student, I want to view the schedule so I know class/exam/event timings.	The schedule list displays event name, date/time, and location for each entry, ordered chronologically by event date. Past events remain visible unless explicitly archived by staff.
US-4	As a student, I want to view staff/TA contacts so I know who to reach.	The contacts list displays name, role, and contact method for each entry. If an office location is provided, it is also shown. Entries are grouped or sortable by role.
US-5	As a student, I want to view resource links so I can find shared materials.	Each resource link entry displays a label and, if provided, a short description. Clicking the label opens the URL in a new tab. Broken or empty URLs are prevented at save time
US-6	As lecturer/TA, I want to create/edit an announcement.	The form requires title, published date, and body before it can be saved. Expiry date is optional. Saving with any required field empty triggers a validation error and does not save the item. Once saved and published, the announcement appears in the

		student-facing list within the same session.
US-7	As lecturer/TA, I want to create/edit an FAQ entry.	The form requires a question, answer, and category before it can be saved. Category must be selected from the existing list (see Tag/category management). Saving with a missing required field shows a validation error and blocks the save.
US-8	As lecturer/TA, I want to create/edit a schedule entry	The form requires the event name, date/time, and location before it can be saved. Description is optional. The date/time field must be a valid date; invalid or empty values block saving with a validation message.
US-9	As lecturer/TA, I want to create/edit a contact entry.	The form requires name, role, and contact method before it can be saved. Office location is optional. Contact method must be a valid email address or phone number format; invalid formats block saving.
US-10	As lecturer/TA, I want to create/edit a resource link.	The form requires a label and URL before it can be saved. The URL must be a valid, well-formed link and malformed URLs block saving with a validation message. Description is optional.
US-11	As lecturer/TA, I want to archive outdated content.	Selecting to archive on any content item will trigger the action of a confirmation dialog. Confirming sets the item's status to archived: it is immediately removed from all student-facing views but remains visible, with full detail, in the administrator/staff content list. Cancelling the dialog leaves the item unchanged.
US-12	As a student, I want to filter content by tag so I only see what's relevant to me.	A tag filter is available on each content list. Selecting one or more tags shows only published items whose target tags include at least one selected tag. Clearing the filter

		returns to the unfiltered, full list.
US-13	As staff/TA, I want to select target tags before publishing.	The content form includes a required multi-select tag field, populated from the current tag list. At least one tag must be selected before the item can be set to published; attempting to publish without a tag triggers a validation error. Saving as draft does not require a tag.
US-14	As an administrator, I want to add/remove tags so the list stays relevant.	The tag management screen lists all existing tags with add, edit, and delete controls, accessible only to the admin role. Adding a tag requires a non-empty, unique name. Deleting a tag prompts confirmation; once deleted, it no longer appears in the selection dropdown for new or edited content, but any content item previously tagged with it keeps that tag unchanged.
US-15	As a user, I want to log in with my department Google account so I can access the system securely.	The login screen offers a single sign in with Google option. On successful Google authentication, the system checks the returned email domain; only addresses ending in @ku.th are granted access. Any other domain is rejected immediately with a clear error message, and no session is created.
US-16	As a user, I want to search content by keyword so I can find information quickly without browsing each section.	A global search field is available from the main navigation. Entering a keyword and submitting returns matching results from announcements, FAQs, schedules, contacts, and resource links in a single combined list, with each result labelled by its content type. Matching is case-insensitive against the title/name and body/description fields of each type. An empty query returns no results and let the user back to enter a search term.

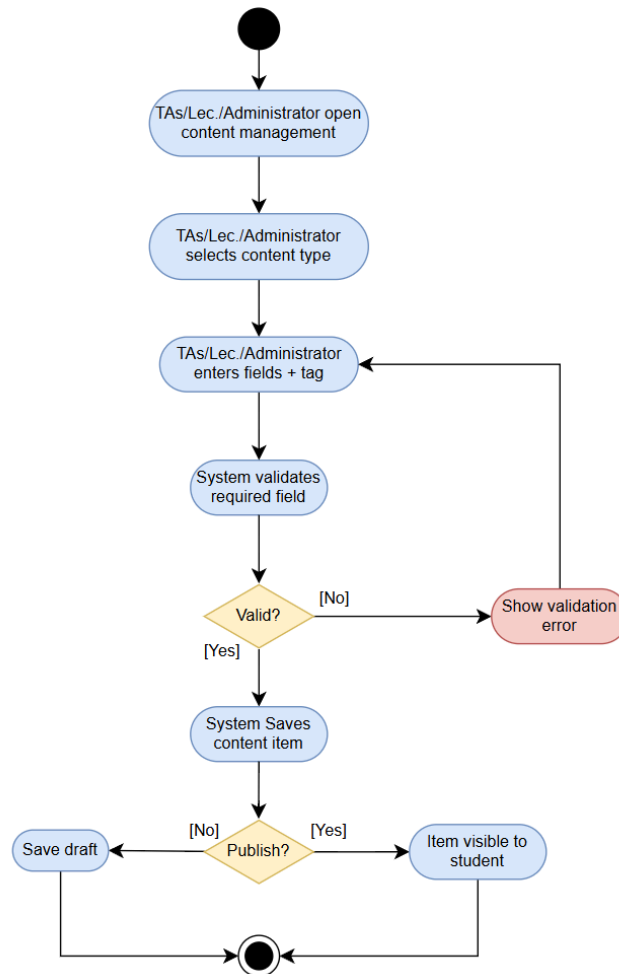
6. User Requirement Specification

ID	User Requirement
URS-1	Students shall be able to view a public list of current announcements.
URS-2	Students shall be able to search or browse FAQ entries.
URS-3	Students shall be able to view the schedule with date, time, and location.
URS-4	Students shall be able to view a list of lecturer/TA/administrator contacts.
URS-5	Students shall be able to view a list of resource links.
URS-6	The system shall allow an authorised lecturer/TA/administrator to create and edit announcements.
URS-7	The system shall allow an authorised lecturer/TA/administrator to create and edit FAQ entries.
URS-8	The system shall allow an authorised lecturer/TA/administrator to create and edit schedule entries.
URS-9	The system shall allow an authorised lecturer/TA/administrator to create and edit contacts.
URS-10	The system shall allow an authorised lecturer/TA/administrator to create and edit resource links.
URS-11	The system shall allow an authorised lecturer/TA/administrator to archive any content item.
URS-12	Students shall be able to filter content by tag.
URS-13	Lecturer/TA/administrator shall be able to select target tags from a dropdown before publishing content
URS-14	An administrator shall be able to add, edit, and remove tags.
URS-15	Users shall be able to log in via Google OAuth using a @ku.th email address.
URS-16	Users shall be able to search content by keyword across all content types.

7. Activity Diagram

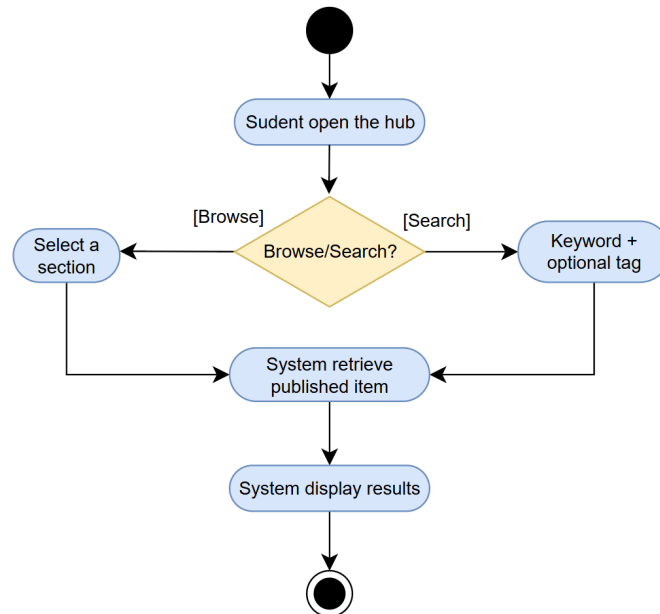
AD-1: Staff/TA creates or updates a content item .

Generic flow applicable to all five content types. Staff opens content management, selects a content type, enters type-specific fields plus at least one target tag, the system validates required fields, shows an error and loops back if invalid, otherwise saves the item and either publishes it or keeps it as a draft. (Derives SRS-3, SRS-4, SRS-5, SRS-6, SRS-9.)



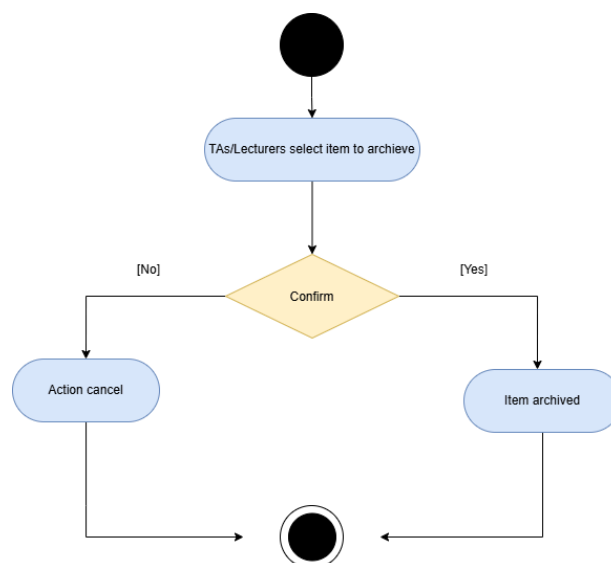
AD-2: Student browses or searches content

The student opens the hub, chooses to browse a section or search by keyword, optionally applies a tag filter, the system retrieves only published items matching the criteria, displays results, the student selects an item, and the system shows its detail. (Derives SRS-1, SRS-2, SRS-10, SRS-14)



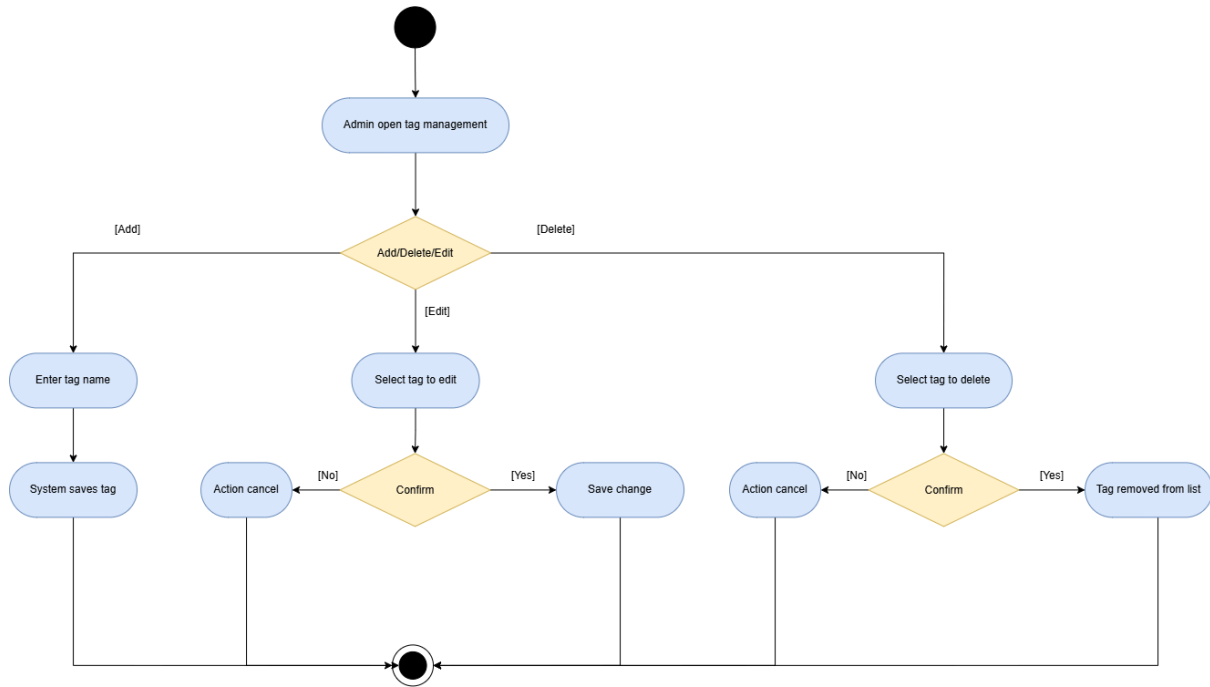
AD-3: Staff/TA/Administrator archives a content item

Staff selects an item to archive, the system asks for confirmation, and if confirmed the item is archived so the information will be hidden from students, and retained in the administrator view, otherwise the action is cancelled. (Derives SRS-8.)



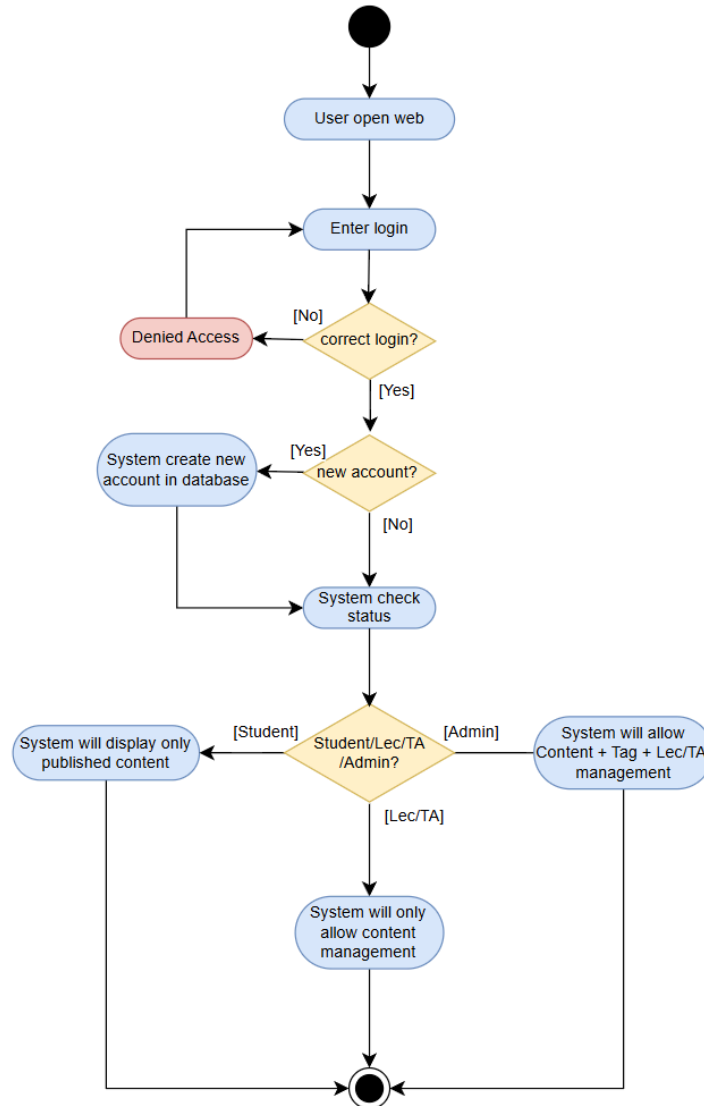
AD-4: Administrator manages tags

Admin opens tag management and chooses to add, edit or delete a tag. To add, the admin enters a tag name and the system saves it, for edit and delete the tag, both the admin starts by selecting a tag then for edit: admin edit name tag while delete do nothing in this stage then the system asks for confirmation, and if confirmed, the tag is removed or edited from the tags of that content. (Derives SRS-11, SRS-12.)



AD-5: User accesses the hub and is routed by role

A user enters the website and login with @ku gmail. The system will check the user's roles: status students view content; Lecturers/TAs see the content management menu; Administrator sees the content management menu, manage tags, and manage Lecturers/TAs. (Derives SRS-1, SRS-3, SRS-11, SRS-13 .)



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall display only published content items to students.
SRS-2	The system shall display each content item's fields according to its type. status of each public menu item.
SRS-3	The system shall allow only authorised staff/TA to create, edit, publish, or archive content.
SRS-4	The system shall require type-specific mandatory fields before saving an item.
SRS-5	The system shall display a validation message and not save the item when required data is missing or invalid.
SRS-6	The system shall store each item with a status of published or draft.
SRS-7	The system shall update the student-facing view to reflect a saved status change.
SRS-8	The system shall hide archived items from students while retaining them in the administrator view.
SRS-9	The system shall require at least one target tag, selected from the existing tag list, before an item can be published.
SRS-10	The system shall support filtering the student-facing content list by tag.
SRS-11	The system shall allow only administrators to create, edit, or remove tags.
SRS-12	The system shall not remove a tag from content already tagged with it, even after that tag is removed from the selectable list.
SRS-13	The system shall permit login only for Google accounts with an email address ending in @ku.th; accounts from other domains shall be rejected.
SRS-14	The system shall support keyword search across all content types, matching against each item's title and body fields.

9. Software Architecture

Based on the SRS for C: Department Information & Communication Hub (Iteration 1 scope: content viewing, content management, tag management, authentication, and search), the recommended architecture is a **Modular Monolithic Architecture using the Model-View-Controller (MVC) pattern**, implemented as a single deployable [Next.js](#) application internally organised into clearly separated layers, rather than a flat monolith or a complex microservices system.

Rationale

The system is being built by a four-person team where most members are new to software development and only one member has prior experience. The current scope covers content viewing across five content types (announcement, FAQs, schedules, contacts, resources links), content management (create, edit, publish, archive), tag-based filtering and search, and authentication restricted to @[ku.th](#) accounts (SRS-13). For a full microservices split would make unnecessary operational, inter-service networking (API Gateway between model), independent deployments, and distributed debugging that a team of mostly first-time developers can not learning and doing within a single iteration, and nothing in the SRS calls for independently scaling individual features or expected load is department-scale or production-scale. A flat, undifferentiated monolith is on the other hand risks to become difficult to maintain as authorisation checks (SRS-3, SRS-11, SRS-13), type-specific validation rules (SRS-4, SRS-5, SRS-9), and search/filter logic (SRS-10, SRS-14) grow across five distinct content types. The modular MVC approach gives the simplicity of a single deployment while separating presentation (View), request handling and authorisation (Controller), and business/data logic (Model), organised into each domain modules (auth, content, tags, search). This keeps each module small enough for members who are beginner to own independently, concentrates the highest-risk logic (OAuth, role checks) in the hands of the strongest developer, and leaves the door open to extracting a module into a microservice later if the hub's scope grows

Component	Layer/Type	Description
Web Application	Presentation (View)	Renders the public content feed (announcements, FAQs, schedules, contacts, resource links) for students, the content management dashboard for staff/TA, and the tag management screen for administrators; consumes data from the Controllers

		and displays confirmations, validation errors, and search/filter results. Errors, and validation messages.
Auth Controller	Controller	Handles the Google OAuth2 sign-in flow, receives the returned account, and rejects any email not ending in @ku.th (SRS-13) before a session is created.
Content Controller	Controller	Receives HTTP requests for each content type (view, create/edit, publish, archive), performs role-based authorisation checks (SRS-3) before any write, and coordinates calls to the Content Model before returning a response to the View.
Tag Controller	Controller	Receives requests to add, edit, or delete tags, restricted to the administrator role (SRS-11), and to attach tags to content items during publishing.
Search Controller	Controller	Receives keyword and tag-filter queries from the View, forwards them to the Search Model, and returns combined, type-labelled results.
Content Model / Business Logic	Model	Encapsulates content business rules per type: validates required fields and formats such as URL or contact method (SRS-4, SRS-5), requires at least one tag before publishing

		(SRS-9), applies published/draft status changes (SRS-6, SRS-7), and applies archive/hide logic (SRS-1, SRS-8).
Tag Model / Business Logic	Model	Manages the tag list, enforces unique tag names, and prevents a deleted tag from being removed from content already tagged with it (SRS-12).
Search Model / Business Logic	Model	Matches a keyword against the title/body fields across all five content types and applies tag filters to published items (SRS-10, SRS-14).
Database	Persistence	Stores content records, the Tag entity, and the Content-Tag junction table; serves queries for the public (published-only) view and the administrator's full record view.
Authentication/Authorization Module	Cross-cutting	Verifies the logged-in user's role (student, staff/TA, administrator) and enforces that only authorised roles can reach content management (SRS-3) or tag management (SRS-11); used by every Controller before processing a write request.

10. Data Storage Technology

The recommended data storage technology is a **Relational Database Management System (RDBMS)**, such as **PostgreSQL**, used as the single data store for this iteration.

Rationale

All data described in the SRS is structured and fits a fixed schema per content type: announcements, FAQs, schedules, contacts, and resource links each have a small set of required fields (SRS-2, SRS-4, SRS-5). The system also requires a many-to-many relationship between content and tags (SG-6, SG-7, SRS-9, SRS-10, SRS-12), which an RDBMS models naturally through a junction table with foreign key constraints a NoSQL store would instead require this relationship to be maintained manually in application code, adding risk for a team with limited prior experience. An RDBMS also enforces required-field and format validation and transactional consistency, directly satisfying SRS-4, SRS-5, SRS-7, SRS-8, SRS-11, and SRS-12. PostgreSQL specifically provides built-in full-text search that, satisfying the keyword-search requirement (SRS-14) without a separate search service and avoiding operational complexity the team does not need to take on. Given the team's limited experience and department-scale load, a relational database keeps the stack simple and integrates cleanly with Prisma.

Component	Data Storage Technology	Description
Content Database	RDBMS (PostgreSQL)	Normalised tables storing content records per type, plus common fields (status, created_by, timestamps); supports SRS-1 through SRS-9.
Tag Database	RDBMS (PostgreSQL)	(PostgreSQL) Tag table and a ContentTag function table recording the many-to-many relationship between content and tags; supports SRS-9 through SRS-12.
User Database	RDBMS (PostgreSQL)	User table storing each authenticated account's email, assigned role (student/staff-TA/admin), and OAuth session reference; referenced by create and update on content and tags, and

		checked before any role-gated action. Supports SRS-3, SRS-11, SRS-13.
Full-text Search Index	RDBMS (PostgreSQL built-in)	tsvector indexes on title/body fields across content types, enabling keyword search in a single query; supports SRS-14.

Only one data store is required for this scope, since content viewing, management, tagging, and search all operate on the same relational schema. A supplementary NoSQL store could be added later if a future iteration introduces less-structured data (e.g. an approval-workflow audit trail), without disrupting this store.

11. Development Environment

The recommended development and deployment environment is a **containerised environment using Docker**, rather than bare metal.

Rationale

The project brief notes that the system must operate across different machines available to the department, such as lab PCs, staff machines, or a hosting platform. A containerised environment packages the Next.js application, its dependencies, and the environment configuration into a portable, reproducible unit, so the four-person team most of whom are new to deployment that can run the exact same setup on each member's laptop during development and on the production host, without manually reconfiguring dependencies each time. Bare-metal deployment would tie the application to the specific configuration of one machine, making it fragile to reproduce across five different development laptops and risky to hand off if the department later moves hosting providers. Docker also makes it easy to redeploy the environment quickly if something breaks, which is valuable given the team has no dedicated DevOps support, and pairs naturally with **GitHub** for version control and CI, so every team member works from the same containerised configuration regardless of their individual machine setup. The **PostgreSQL database is hosted on Supabase** rather than run as a local container in production, which removes the operational burden of managing database backups, scaling, and uptime tasks a beginner-heavy team is not well positioned to handle manually while still exposing a standard PostgreSQL connection that Prisma can use identically in development and production.

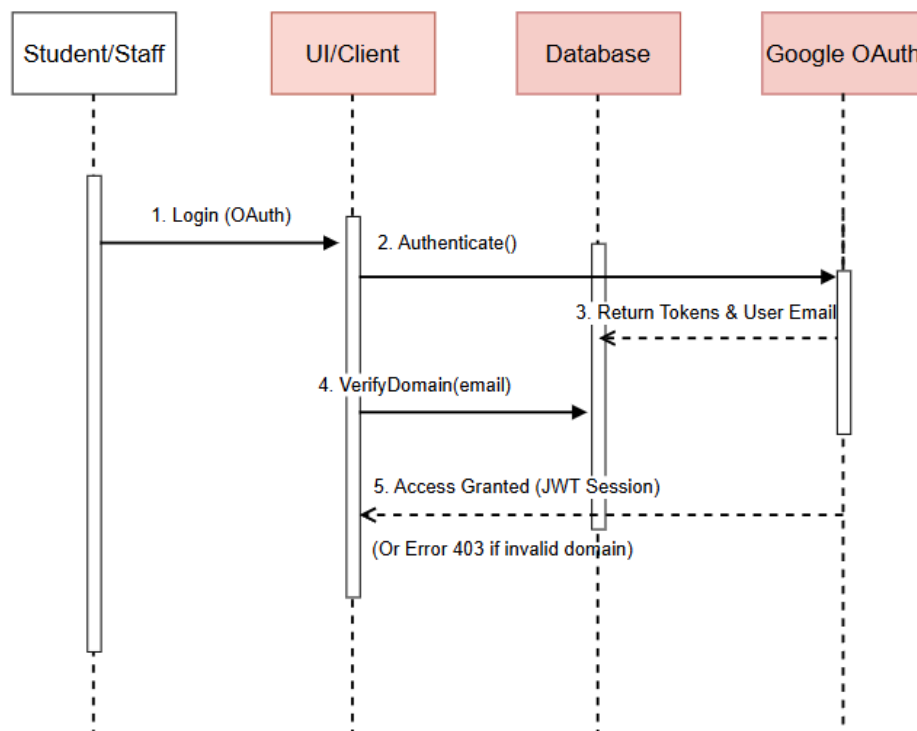
Component	Development Environment	Description
Application Server (Next.js: Controllers + Models)	Docker container version 29.7.2	Runs the Next.js application (API routes and business logic) in an isolated, reproducible container image built from a single Dockerfile, deployable identically on a developer's laptop or the department's hosting platform.
Content/Tag Database	Supabase (managed PostgreSQL)	Hosts the relational database remotely, so the team does not need to self-manage a database server; the same connection string is used by Prisma in both development and production.
Version Control / CI	GitHub	Stores the codebase, tracks changes across all four members, and can run automated checks (e.g., type-checking, tests) before code is merged.
Local Development Machine	Docker Desktop	Used by each team member to build and run the application container locally, connecting to either a local or a shared Supabase instance during development, mirroring the production container configuration to minimise environment inconsistencies.
Styling	Tailwind CSS (bundled in the app container) version 4.3.3	Utility-first CSS compiled as part of the Next.js build; no separate service or container required.

12. Sequence Diagrams for All SRS

The following sequence diagrams are derived from the chosen Modular Monolithic MVC architecture and cover the core interactions traced in the SRS traceability matrix: authentication, content creation/editing, content publishing, tag management, and student-facing viewing/filtering/search. .

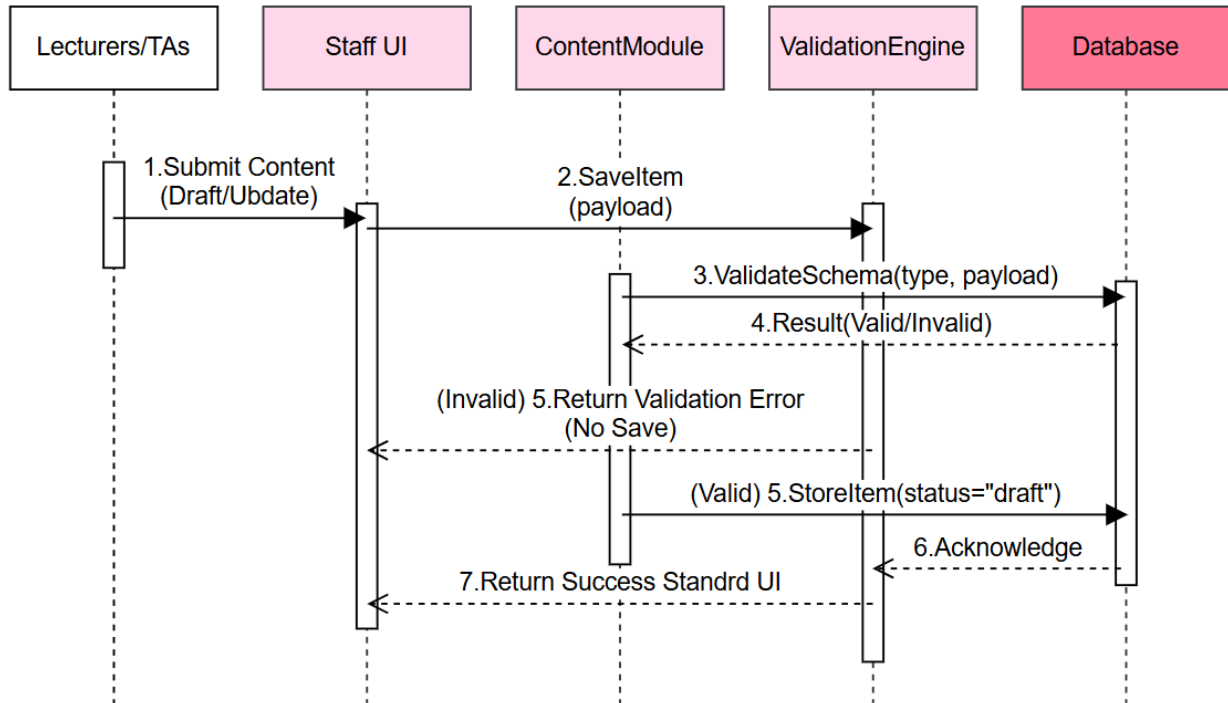
SQD-1: Authentication & Authorization

Traced requirements: SRS-13



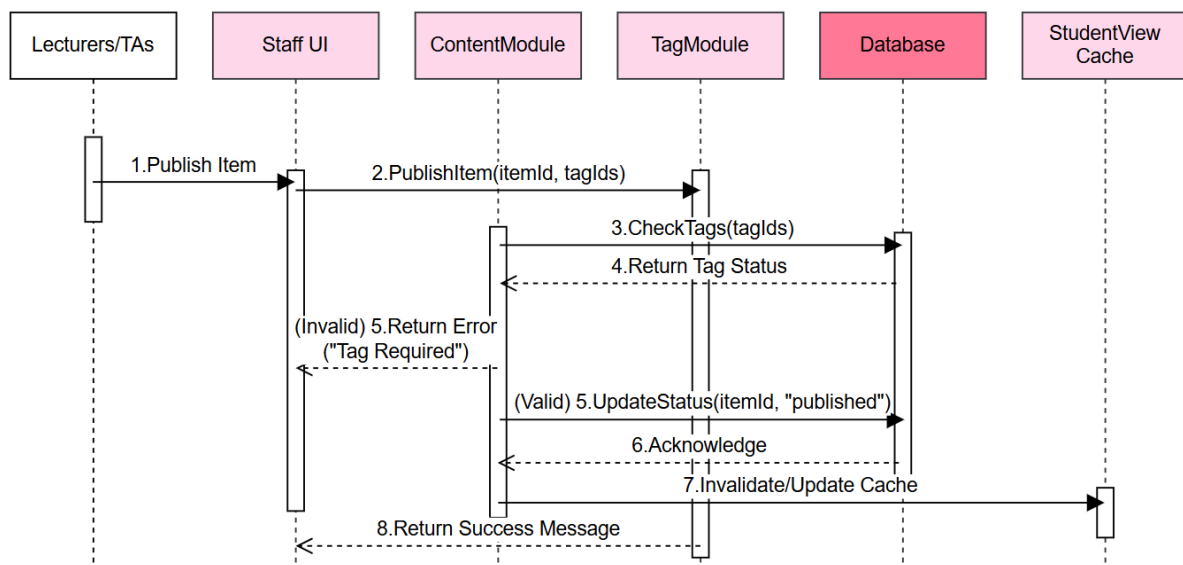
SQD-2: Create / Edit & Validate Draft Content

Traced requirements: SRS-3, SRS-4, SRS-5, SRS-6



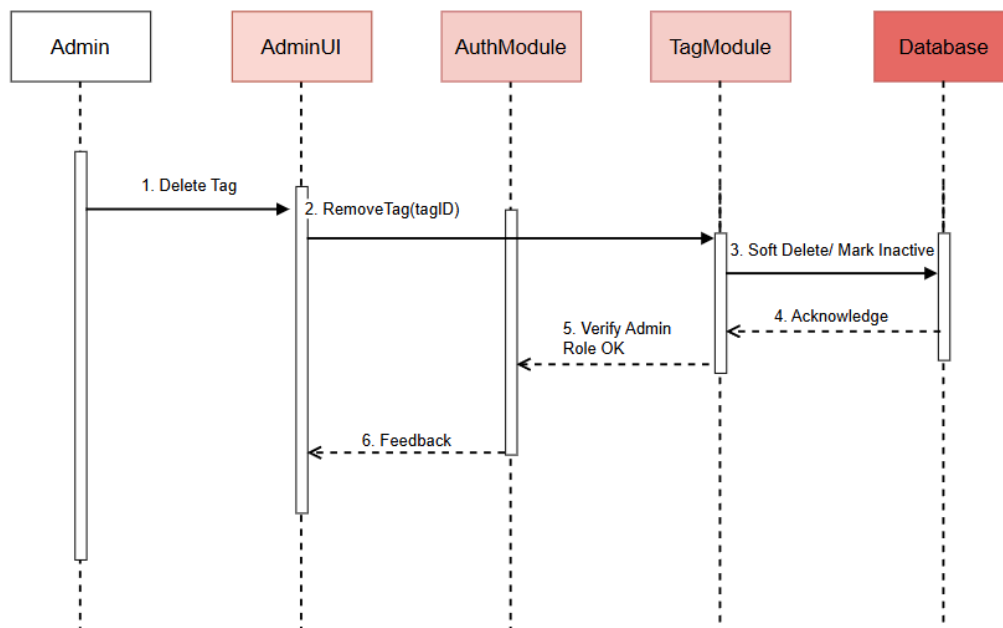
SQD-3: Publish Content & Enforce Target Tags

Traced requirements: SRS-3, SRS-6, SRS-7, SRS-9



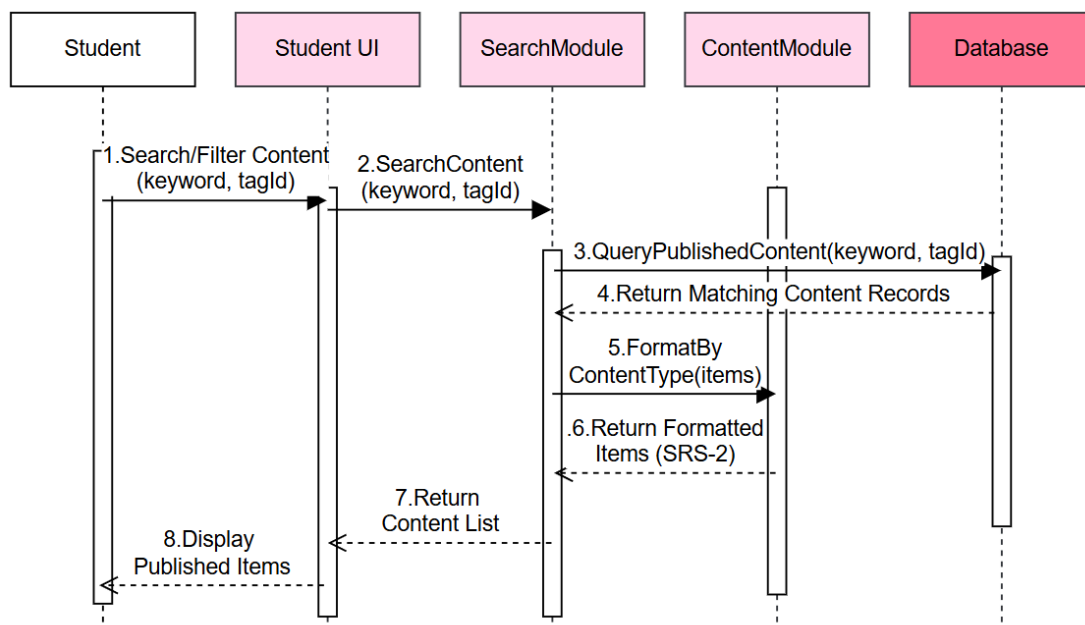
SQD-4: Admin Management & Tag Lifecycle

Traced requirements: SRS-11, SRS-12



SQD-5: Student Content Viewing, Filtering, & Search

Traced requirements: SRS-1, SRS-2, SRS-8, SRS-10, SRS-14



13. Traceability Matrix

From Sub-Goal to SRS

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	View announcements	US-1	URS-1	AD-2	SRS-1, SRS-2
SG-1	View FAQ	US-2	URS-2	AD-2	SRS-1, SRS-2
SG-1	View schedule	US-3	URS-3	AD-2	SRS-1, SRS-2
SG-1	View contacts	US-4	URS-4	AD-2	SRS-1, SRS-2
SG-1	View resource links	US-5	URS-5	AD-2	SRS-1, SRS-2
SG-2, SG-3	Create/edit content	US-6-US-10	URS-6-URS-10	AD-1	SRS-3, SRS-4, SRS-5, SRS-6, SRS-7
SG-4	Prevent invalid content at entry	US-6-US-10	URS-6-URS-10	AD-1	SRS-4, SRS-5
SG-4	Remove/hide outdated content	US-11	URS-11	AD-3	SRS-1, SRS-8
SG-5	Archive content	US-11	URS-11	AD-3	SRS-8
SG-6	Filter by tag	US-12	URS-12	AD-2	SRS-10
SG-6, SG-3	Select target tag on publish	US-13	URS-13	AD-1	SRS-9
SG-7	Manage tags	US-14	URS-14	AD-4, AD-5	SRS-11, SRS-12
SG-8	Login via Google (@ku.th)	US-15	URS-15	AD-5	SRS-13
SG-9	Search content by keyword	US-16	URS-16	AD-2	SRS-14

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-5
SRS-2	SQD-5

SRS-3	SQD-2, SQD-3
SRS-4	SQD-2
SRS-5	SQD-2
SRS-6	SQD-2, SQD-3
SRS-7	SQD-3
SRS-8	SQD-5
SRS-9	SQD-3
SRS-10	SQD-5
SRS-11	SQD-4
SRS-12	SQD-4
SRS-13	SQD-1
SRS-14	SQD-5